Hasso Plattner Institut
Digital Engineering · Universität Potsdam

Programmiertechnik II

Leonhard Hennicke

Tutorium: Git, GitHub, Iterators, File Streams, Debugging

Überblick

1. Git, GitHub, Classrooms
   - Install & Register
   - Basic Commands
   - Merging
   - **`Pull Requests`**
2. Iteratoren
3. File Streams

Install & Register

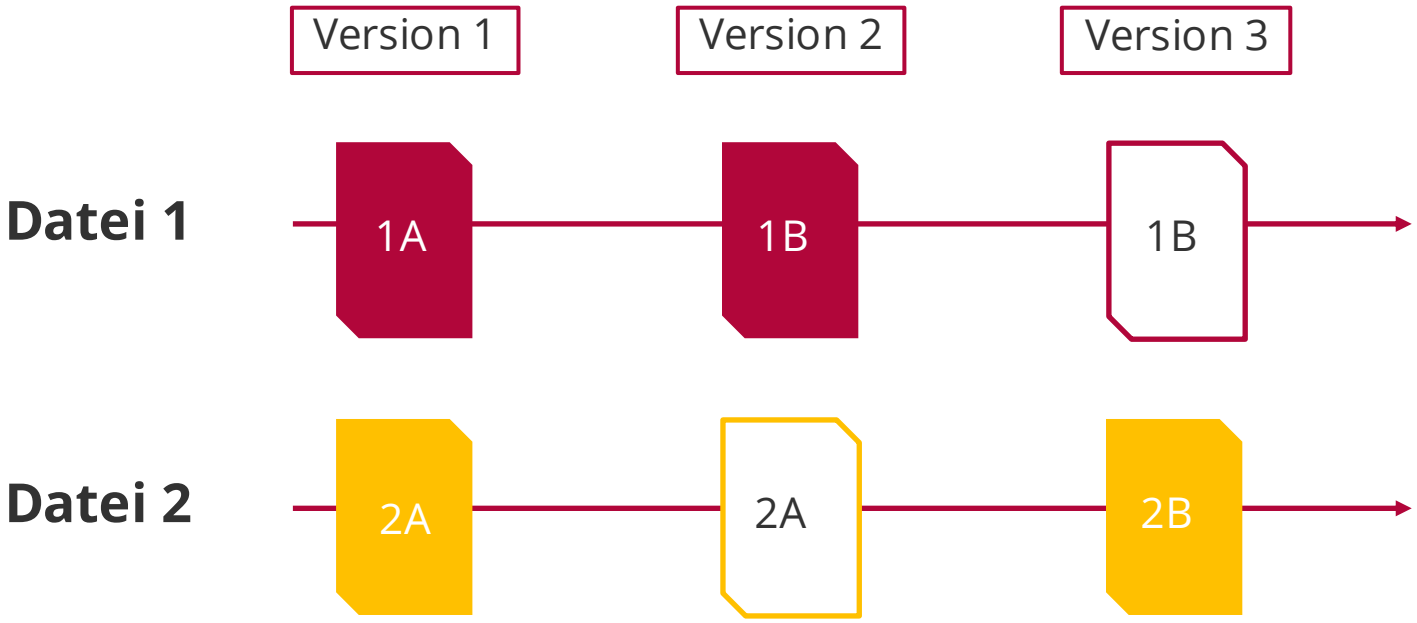
1. Git installieren
   - https://git-scm.com/downloads
2. GitHub Account erstellen
   - https://github.com
3. GitHub Classroom Assignment Link
   - https://classroom.github.com/a/78sXz8Oi
4. Detailliertere Tutorials zu Git (optional)
   - https://www.atlassian.com/git

Git vs. GitHub

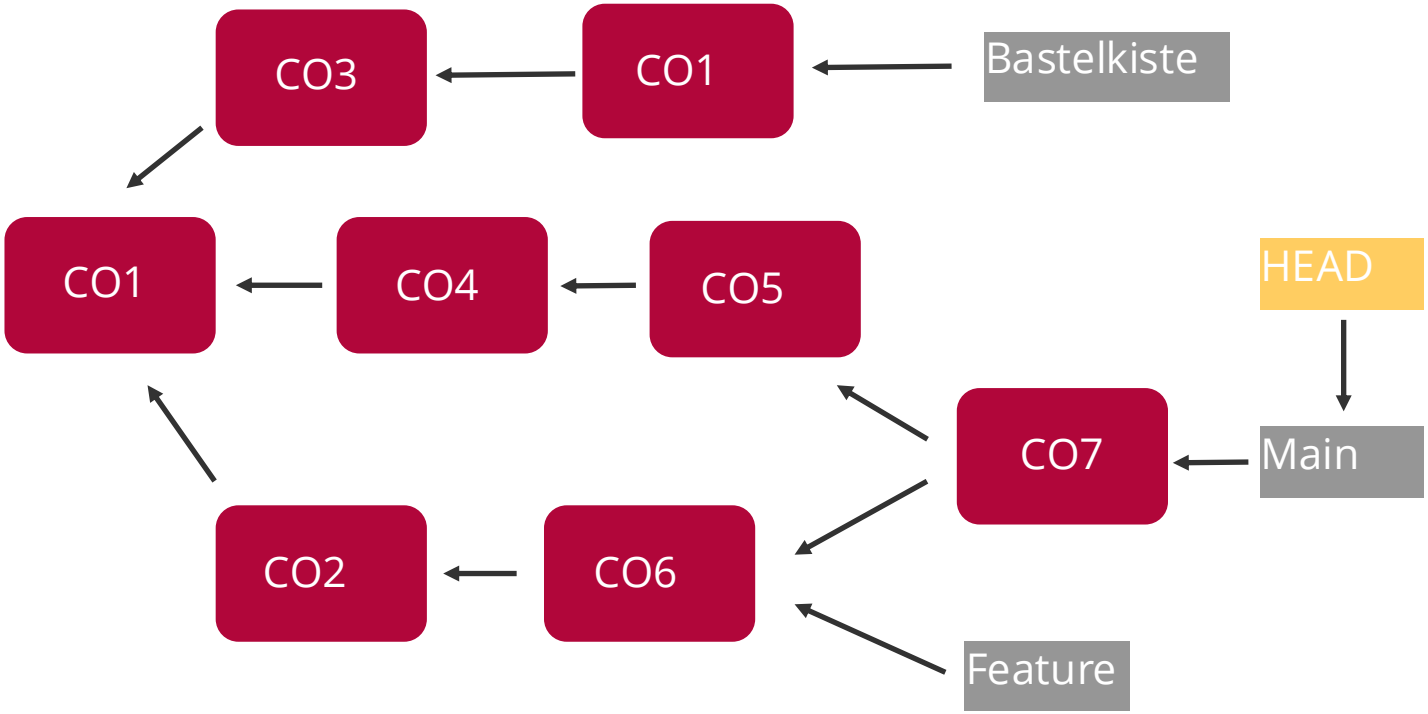
1. Git
   - System zur verteilten Versionsverwaltung
   - Open Source
   - Ursprünglich entwickelt von Linus Torvalds in 2005

2. GitHub
   - Proprietäre Softwareentwicklingsplatform
   - Betrieben von Microsoft
   - Basierend auf Git

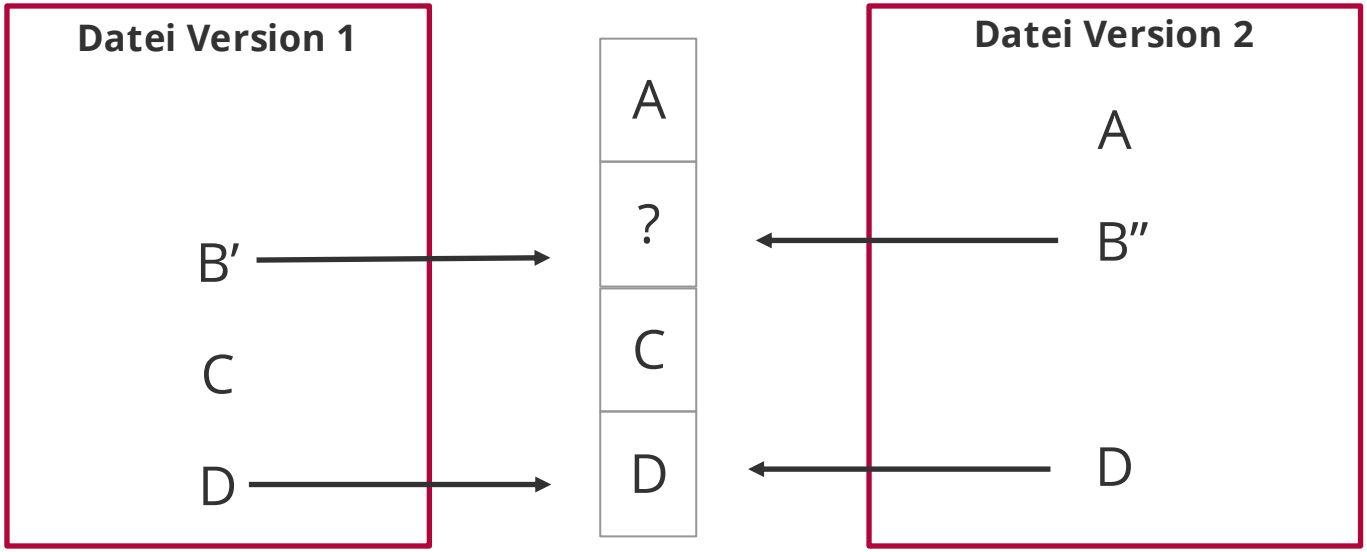
Git



Git: Branches



Merge Konflikte



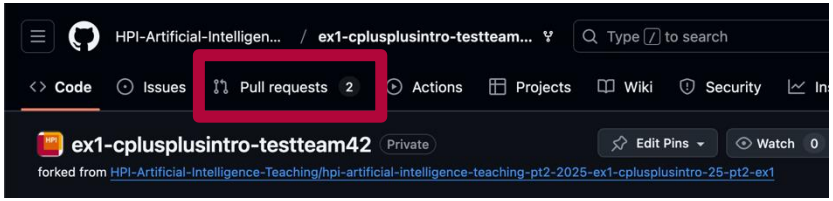
Original A B D

Git: Befehle

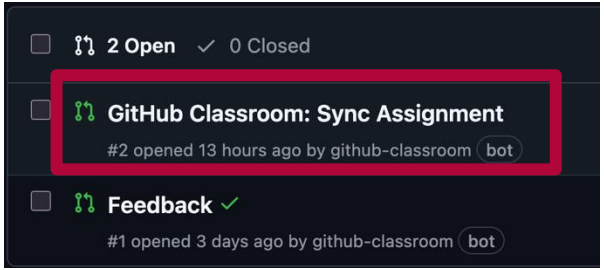
- git init
- git help, git help <befehl>
- git clone <repo link>
- git status, git log, git diff
- git pull
- git add <file-Pfad>, git add -A
- git commit -m “<commit message>”
- git push
- git checkout -b <new branch name>
- git checkout <branch name>
- git merge <branch name>

Pull Requests im Übungsrepo

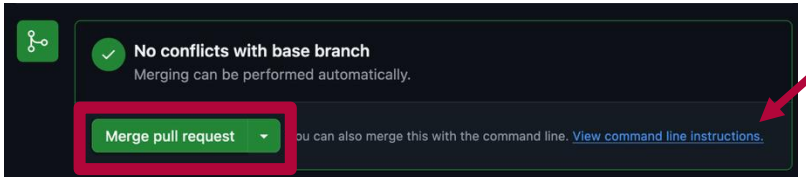
1.



2.



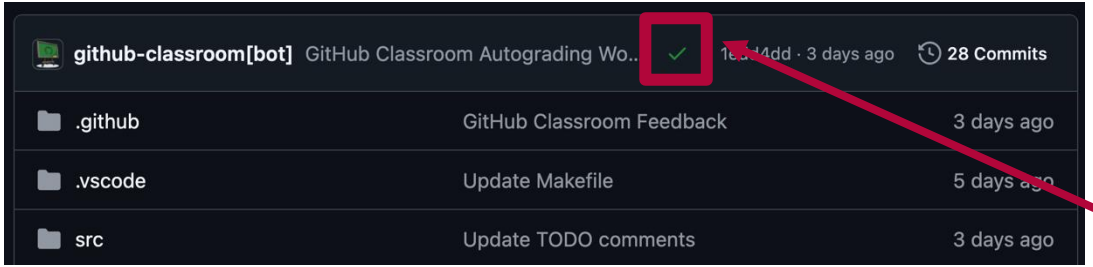
3.



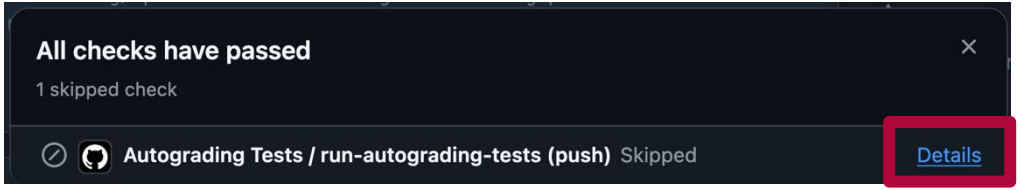
Lokal mergen ist auch eine Option

GitHub Classrooms: Autograding

1.



2.



Automatisierte Tests via GitHub Classrooms nach jedem Push

Iterator

- **Problem:** Selbst definierte Klasse soll einfach iterierbar sein
- **Lösung:** Eine zweite selbst definierte Klasse, die sich verhält wie Iteratoren anderer Container-Typen (z.B. vector::iterator), definiert Logik, wie iteriert werden soll
- **Vorteile:**
  - Range-basierte for-Loops: *for (auto it : MyContainer) {}*
  - STL Algorithmen für ander Container funktionieren auf der neuen Klasse
- **Minimales Interface:** Operatoren ++, *, ==, !=, Definition von value_type, sowie von begin() und end()

array_iterator.h, array_iterator.cpp

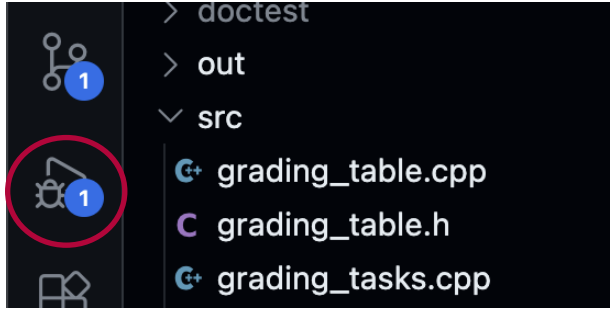
File Stream (std::ifstream)

- Standard C++ class um Daten aus Dateien zu lesen
- Öffnet eine Datei und behandelt sie als Input Stream aus Characters/Bytes
- Binärdaten können mit .read() gelesen werden
- Um andere Datentypen zu lesen, sagen wir der .read() Methode, dass die so tun soll, als wären die Variablen, in die eingelesen wird, char Pointer, ohne etwas an den eigentlichen Daten zu ändern
- Um Daten in Variable x einzulesen:
  - my_filestream.read(**reinterpret_cast<char*>(&x)**, sizeof(x));

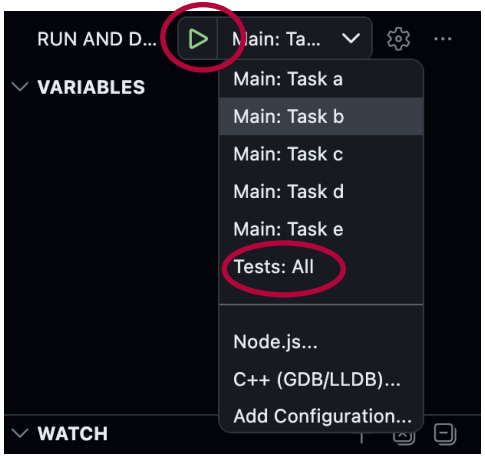
filestreamin.cpp

Debugger

1.


doctest
out
src
grading_table.cpp
grading_table.h
grading_tasks.cpp

2.


RUN AND D...
Main: Ta...
VARIABLES
Main: Task a
Main: Task b
Main: Task c
Main: Task d
Main: Task e
Tests: All
Node.js...
C++ (GDB/LLDB)...
Add Configuration...
WATCH

Debugger

3.

RUN AND D... Main: Ta...

VARIABLES
Locals
task = 97 'a'
fp = {...}
table = {...}
agent = 0x0
argc = 3
argv = 0x7fffffffcf18
Registers

WATCH
task = 97 'a'
task == 'b' = false

CALL STACK Paused on breakpoint
main(int argc, char ** argv) 15:1

BREAKPOINTS
All C++ Exceptions
main.cpp src 15

[Preview] readme.mc

src > main.cpp > main(int, char * [])

```
1  #include <iostream>
2  #include <fstream>
3
4  #include "grading_tasks.h"
5  #include "grading_table.h"
6
7  /* main entry point for the program */
8  int main(int argc, char *argv[]) {
9      if (argc < 3) {
10         std::cerr << "Usage: " << argv[0] << " (a|b|c|d|e) [filename]" << '\n';
11         return 1;
12     }
13
14     char task = argv[1][0];
15     std::ifstream fp{argv[2], std::ios::binary};
16     if (!fp) {
17         std::cerr << "Failed to open file: " << argv[2] << '\n';
18         return 1;
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Filter (e.g. text, !exclude, \e...

```
Loaded '/lib/x86_64-linux-gnu/libstdc++.so.6'. Symbols loaded.
Loaded '/lib/x86_64-linux-gnu/libm.so.6'. Symbols loaded.
Loaded '/lib/x86_64-linux-gnu/libgcc_s.so.1'. Symbols loaded.
Loaded '/lib/x86_64-linux-gnu/libc.so.6'. Symbols loaded.

Breakpoint 2, main (argc=3, argv=0x7fffffffcf18) at src/main.cpp:15
15          std::ifstream fp{argv[2], std::ios::binary};
Execute debugger commands using "-exec <command>", for example "-exec info registers" will list registers in
use (when GDB is the debugger)
```

Codespaces: laughing giggle | main* | 0 0 | 0 | Main: Task a (ex1-cplusplusintro-testteam-pt2-26) | UTF-8 | LF | { } C++ | Finish Setup | Layout: U.S. | Linux

Hasso Plattner Institut
Digital Engineering · Universität Potsdam

Viel Spaß bis zur nächsten Vorlesung!